Semantic Capability Engines

Luc Alexander Lüthi

January 28th, 2026

Contents

1 Introduction

The initial goal for the semantic effect engine was to provide a base substrate for a sort of higher order vulnerability research, wherein you could use the models generated by the system to study classes of vulnerabilities, how they arise, and how to constraint your software to avoid them.

2 Initial Semantic Model through Flat Logic Graphs

Reducing our programs to their vulnerable interfaces, we revieve purely how they interact with the world. The actual computation occuring in the modeled software is not relevant, making these true models and definitively not machines. This process reduces a program, or other securable system, to a series of events in a protocol consisting of flavored reads and writes. This base model is surprisingly enough not abstract enough to really get any kind of reasoning over permissions, access, or capabilities in a general sense. There is also no sense of conditionality which while not computationally required makes it much more straight forward to reason about a system as a human, and makes computational representation more straight forward. The first problem however is the notion of semantic intent behind a model, which must be made clear so that what is actually being secured can itself be reasoned about and protected.

Secured systems are arguments. They present judgements, and affirm that those judgements are fact. In doing this however they make assumptions about the proof of the judgements being argued. The mismatch between what a system assumes is true as part of a proof of its judgements, and what is actually true, is where atackable vulnerabilities hide. Every system can be represented by an isolated logical universe. This universe contains predicates and rules over these predicates. A simple security judgement emulating a real world security scenario, unauthorized access, can be modeled by a judgement that state B is unreachable given state A. An exploiting entity is tasked with proving that B is in fact reachable and that the assumptions made by the system are not enforced by the actual model. When we reason about this in simple first order logic, we can model pretty much any system with a sufficiently complex directed graph of the flow of logical predicates in a systems logical universe. It is trivial to find logical

inconsistencies in a graph of this kind, as finding a vulnerability reduces to finding a path between initial and unauthorized states. Real systems and the security judgements made over them, are categorically more intricate, and do not have public judgements.

The state, truthfully, of real systems is much more complex than a single predicate. Surely if we were to collapse every possible state of a system to a single predicate, and did so for every possible combination of relevant state in the system, we could find a flat graph, but this space is combinatorially hard.

3 Capability Hypergraphs

In order to retain the ability to analyze the system, the model will remain public to the exploiting entity whether it be a researcher or a modeled attacker. States will be designated as a capability on a user, users may achieve any capability as long as the rules of the universe lead to it. The rules in our logical universe will follow a scheme that turns our flat logical directed graph into a hypergraph, where edges are rules between complex states. Let the environment be a set of currently applicable states. Each rule has a requirement state set, a set of states it consumes when invoked, and a set of states it introduces into the new environment when invoked.

$$S^{i+1} = S \cup \text{invoked} - \text{consumed}$$

Judgements in real systems are typically inferred based on goals, so we will designate a target state, a singular capability on a user. We will also designate a starting set of states, assumed to be an unprivileged entity in the logical universe. The argument may now commense, where an attacker is interested in proving that a capability is reachable on the unprivieged user, and a defender of the argument judgement is interested in preventing that from happening all using the logical rules in the system. Both an attacker and defender may make moves in this exercise, both progressing the same environment state set. They are not traversing a complex search space. An ideal move can be conceptualized for each arguing entity as the next rule invocation which either maximizes or minimizes the steps required to reach the goal capability for the unprivileged user. It is also possible for a move in the space to be made which either makes it no longer possible to prevent reaching the target capability, or makes it impossible to reach the capability.

4 Generation

Opinionated algorithmic generation of problems which result in interesting or studyable scenarios is unrealistic. It was decided to generate these problems through heuristic fuzzing to allow for a more broad search space of systems.

After randomizing a set of capabilities, users, an initial environment, and rules between these states form a pool of candidate systems, we take the Shannon entropy of the proposed system. Whichever candidate is closest to a theoretical continuous median of the set of candidates is taken for further heuristic evaluateion. Evaluating this entropy consists of performing N random steps from the initial environment state and counting how many times each state is reached.

$$P(s) = \frac{\text{count}(s)}{N}$$

$$\text{shannon}(s) = \sum P(s) \log_2 P(s)$$

A target state is then determined. This is done by taking the least occuring state in the process of computing Shannon entropy. Any system for which the target state is present in the initial state or the number of logical steps found to reach the target state unimpeeded by a defending entity is less than a given parameter is discarded. Any system where the number of unique paths to the target state is greater than a given maximum path parameter is also discarded.

The final heuristic at present consists of a proof argument simulation, an exploiting entity is allowed to make two moves in the interest of proving a judgement over the target state reachable, followed by one defending move. This process is repeated until either the number of steps exceeds a given parameter or the target state is reached. All models where the minimum number of steps found to reach the target state is above a given threshold are discarded.